Node.js ja Express

11IT · Veebirakendused · seitsmes tund · konspekt

> **Selle tunni suur mõte:** backend ei ole võlukast kuskil kaugel andmekeskuses. See on tavaline JavaScripti programm, mis käivitub, jääb ootama ja vastab igale päringule, mis tema poole tuleb. Täna kirjutad sellise ise.

Kes seal teisel pool vastab?

API tunnis küsisid andmeid PokeAPI-lt. Sinu `fetch` läks kuhugi teele ja natukese aja pärast tuli JSON tagasi. Aga kes seal vastas? Mitte inimene ja mitte andmebaas ise. Vastas programm, mille keegi on kirjutanud umbes nii: „kui keegi küsib aadressi `/api/v2/pokemon/pikachu`, otsi Pikachu andmed üles ja saada need JSON-ina tagasi“.

Selline programm käitub teisiti kui kõik, mida oled seni kirjutanud. `node tere.js` trükkis kaks rida ja lõpetas, terminal oli kohe jälle vaba. Server ei lõpeta. Ta käivitub, ütleb „olen valmis“ ja jääb ootama. Esimene päring võib tulla sekundi pärast või tunni pärast. Kui see tuleb, teeb server oma töö ära ja ootab edasi.

Seda oled tegelikult juba näinud. Pärast `npm run dev` ei andnud terminal `PS …>` rida tagasi, sest Vite on ka server ja ta jäi ootama. `Ctrl+C` paneb serveri kinni. Restorani keeles: köök on lukus. Kelner võib tellimusi tuua palju tahab, keegi ei vasta.

Node.js: JavaScript ilma brauserita

Seni elas su JavaScript brauseris. Node.js on programm, mis käivitab JavaScripti otse arvutis. Keel on täpselt sama: muutujad, funktsioonid, `if`, massiivid ja `map` töötavad nagu ennegi. Muutunud on see, mis ümberringi on.

	Brauseris	Node’is
Leht, nupud, `document`	on olemas	puuduvad, Node’il pole ekraani ega lehte
`console.log`	läheb F12 konsooli	läheb terminali
Failid arvutis	ei pääse ligi (turvalisuse pärast)	saab lugeda ja kirjutada
Võrk	saab päringuid saata (`fetch`)	**saab päringuid saata ja ka vastu võtta**
Kes käivitab	kasutaja, kui lehe avab	sina, käsuga `node fail.js`

Kõige olulisem on tabeli neljas rida. Node oskab päringuid vastu võtta ehk olla server. Seepärast saab backendi kirjutada samas keeles, mida juba oskad.

npm ja paketid

Serveri saaks kirjutada puhta Node’iga, aga see on üsna tülikas. Kõik kirjutavad sama koodi: loe päringust aadress välja, otsusta, mida teha, pane vastusele õiged päised. Nii et keegi kirjutas selle ühe korra valmis ja pani npm-i üles. Sellist valmis koodi nimetatakse **paketiks**. Meie kasutame paketti nimega **Express**, see on Node’i serverite seas kõige levinum.

```
npm install express
```

See üks käsk teeb kolm asja:

- laeb Expressi alla kausta `node_modules`. Koos Expressiga tuleb kaasa mitukümmend teist paketti, mida Express ise vajab. Ära ehmu, kui seal on palju kaustu;
- kirjutab `package.json`-isse rea `"express": "^5.3.0"` (versioon võib olla teine). See on nimekiri: „sellel projektil on vaja Expressi“;
- teeb faili `package-lock.json`, kuhu on kirjas kõigi pakettide täpsed versioonid.

Tänu sellele nimekirjale ei pea `node_modules`-it kellelegi saatma ega Gitti panema. Kes iganes projekti saab, kirjutab `npm install` ja npm laeb samad paketid ise alla. Reacti projektiga tegid sa täpselt sama asja, lihtsalt siis juhendi järgi.

> **Õige kaust!** Söökla projektis on nüüd kaks `package.json`-it: üks projekti juurkaustas (frontend) ja teine kaustas `server` (backend). Express kuulub backendile. Enne installimist kirjuta `cd server` ja kontrolli `pwd`-ga, kus oled.

Esimene server

Terve server, mis vastab ühele aadressile, mahub kümnesse ritta:

`server/index.js`

```
import express from 'express';

const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Tere! Söökla server töötab.');
});

app.listen(PORT, () => {
  console.log(`Server töötab: http://localhost:${PORT}`);
});
```

Kood	Mida see teeb
`import express from 'express';`	Toob Expressi paketist sisse. Sama `import`, mida kasutasid Reactis, aga Node vajab selleks `package.json`-isse rida `"type": "module"`.
`const app = express();`	Teeb uue serveri. `app` ongi meie server, kõik reeglid paneme talle.
`app.get('/', (req, res) => {`	Reegel: kui keegi teeb GET-päringu aadressile `/`, käivita see funktsioon.
`res.send('Tere! ...');`	Saadab vastuseks teksti.
`app.listen(PORT, () => {`	Käivitab serveri pordil 3000 ja jääb päringuid ootama. Funktsioon käivitub üks kord, kui server on valmis.

Käivita `node index.js` ja ava brauseris `http://localhost:3000`. Ekraanil on sinu tekst. Sinu brauser saatis päringu, sinu programm vastas.

Marsruut: meetod, aadress ja funktsioon

Expressi server on sisuliselt reeglite nimekiri. Iga reegel ütleb: kui tuleb *selline* päring *sellele* aadressile, tee *seda*. Ühte sellist reeglit nimetatakse **marsruudiks** (ingl *route*).

```
app.get('/api/arvustused', (req, res) => {
  res.json(arvustused);
});
```

Tükk	Mis see on
`app.get`	Meetod. `get` vastab GET-päringutele. Järgmises tunnis tuleb juurde `app.post`.
`'/api/arvustused'`	Aadress, millele see reegel vastab.
`(req, res) => { … }`	Funktsioon, mille Express käivitab iga sellise päringu peale.
`req`	**Request** ehk päring: kõik, mida klient küsis (aadress, parameetrid, saadetud andmed).
`res`	**Response** ehk vastus: sellega saadad kliendile midagi tagasi.

Vastuse saatmiseks on mõned käsud:

Käsk	Mida teeb
`res.send('Tere')`	Saadab teksti.
`res.json(andmed)`	Saadab andmed JSON-ina. Teeb ise `JSON.stringify` ja ütleb brauserile, et tuleb JSON. Seda kasutame kõige rohkem.
`res.status(404).json({ … })`	Paneb vastusele staatusekoodi ja saadab siis JSON-i. Kui staatust ei pane, on see 200.

> **Igale päringule täpselt üks vastus.** Kui funktsioon ei jõua kunagi `res.json`-i ega `res.send`-ini, ei saa brauser midagi ja ootab. Ring keerleb, leht laeb ja laeb. Viga ei tule, sest server ei tea, et ta midagi unustas. Kui leht jääb laadima, otsi kõigepealt puuduvat vastust.

Aadressi osad

Võtame ühe aadressi tükkideks:

```
http://localhost:3000/api/arvustused/2?min=4
```

Osa	Nimi	Tähendus
`http://`	protokoll	kuidas räägitakse
`localhost`	host	mis arvutiga. `localhost` on see arvuti, kus sa ise istud
`3000`	port	millise programmiga selles arvutis
`/api/arvustused/2`	tee	selle järgi valib Express, milline marsruut käivitub
`?min=4`	päringu parameetrid	lisasoovid, mitu tükki eraldatakse `&`-ga: `?min=4&sort=toit`

Ühe arvustuse jaoks ei tee keegi eraldi marsruuti `/api/arvustused/1`, `/api/arvustused/2` ja nii edasi. Selle asemel pannakse aadressi muutuv osa, mille ees on koolon:

```
app.get('/api/arvustused/:id', (req, res) => {
  console.log(req.params.id);   // aadressi /api/arvustused/2 puhul: '2'
});
```

Koolon tähendab: siin võib olla mida iganes, pane see `req.params.id`-sse. Päringu parameetrid `?min=4` jõuavad samamoodi `req.query.min`-i.

> **Lõks, millesse peaaegu kõik esimesel korral astuvad:** aadress on alati tekst. `req.params.id` on `'2'`, mitte `2`. Ja `'2' === 2` on JavaScriptis `false`. Kui võrdled seda arvuga, ei leia sa kunagi midagi. Tee enne `Number(req.params.id)`.

Staatusekoodid

Need tunned juba frontendi ja backendi konspektist. Nüüd paned neid ise:

Kood	Tähendus	Millal sina selle saadad
200	OK	kõik läks hästi. `res.json` paneb selle ise
201	Created, loodud	uus asi lisati. Järgmises tunnis, kui tuleb POST
400	Bad Request, vigane päring	klient saatis jama, näiteks hinde 7
404	Not Found, ei leitud	sellise id-ga arvustust pole
500	Server Error	seda sa ise ei saada. See tuleb siis, kui sinu kood jookseb kokku

Üks 404 tuleb ka ilma sinuta. Kui ükski marsruut aadressile ei sobi, vastab Express ise: `Cannot GET /api/arvustus`. See on Expressi teade, mitte sinu oma. Tavaliselt tähendab see kirjaviga aadressis või marsruudis.

Kaks serverit, kaks porti

Söökla projektis töötab nüüd korraga kaks programmi. Neid on lihtne segi ajada, nii et siin on nad kõrvuti:

	Frontend	Backend
Kaust	projekti juurkaust	`server`
Käivitamine	`npm run dev`	`npm run dev` (kaustas `server`)
Programm	Vite	Node + Express
Aadress	`localhost:5173`	`localhost:3000`
Mida annab	HTML, CSS ja Reacti kood	andmed JSON-ina

Mõtle pordist kui korteri numbrist. Maja on sama (`localhost`), aga uksed on eri numbritega ja iga ukse taga elab eri programm. Ühe ukse taga saab elada ainult üks. Kui proovid teist serverit samale pordile panna, saad vea `EADDRINUSE`.

Täna need kaks veel omavahel ei räägi. Backend vastab ainult brauseri aadressiribale.

node --watch

Server loeb su koodi sisse üks kord, käivitamisel. Kui muudad faili ja salvestad, töötab edasi vana kood. Pead serveri `Ctrl+C`-ga kinni panema ja uuesti käivitama. Seda unustab igaüks vähemalt korra ja otsib siis pool tundi viga, mida tegelikult enam pole. `node --watch index.js` teeb taaskäivituse ise iga salvestuse peale. Paneme selle `package.json`-isse skriptiks `dev`, siis on käsk sama mis frontendil: `npm run dev`.

Kui terminal ütleb midagi hirmsat

Backendi vead ei tule F12 konsooli, vaid **terminali**, kus server töötab. Hoia see silma all.

Teade	Mida see tähendab	Mida teha
`EADDRINUSE` … `:3000`	Port 3000 on juba kellegi käes. Tavaliselt sinu enda server teises terminalis.	Leia see terminal ja vajuta `Ctrl+C`.
`Cannot find package 'express'`	Selles kaustas pole Expressi paigaldatud.	`cd server`, siis `npm install express`.
`Cannot use import statement outside a module`	Node ei tea, et kasutad `import`-i. Uuem Node annab vea asemel hoiatuse `MODULE_TYPELESS_…` ja töötab edasi.	`package.json`-isse `"type": "module"`.
`Cannot GET /api/…` (brauseris)	Sellist marsruuti pole.	Võrdle aadressi ja `app.get` rida täht-tähelt.
`ERR_CONNECTION_REFUSED` (brauseris)	Server ei tööta.	Vaata terminali: kas server käib või kukkus veaga kokku?
Leht laeb ja laeb	Marsruut ei saatnud vastust.	Kontrolli, kas kood jõuab `res.json`-ini.
`SyntaxError`	Kirjaviga koodis.	Terminal näitab faili nime ja rea numbrit. Mine sinna.

Mis edasi?

Täna vastab sinu server brauseri aadressiribale. Järgmises tunnis paneme Reacti küsima arvustusi sinu serverilt ja fail `andmed.ts` läheb prügikasti. Siis tuleb ka `POST`: uus arvustus läheb serverisse ja server otsustab, kas hinne 7 on lubatud. Spoiler: ei ole.